

GUÍA DE ACTIVIDAD PRÁCTICA

Construcción de una API REST

con Onion Architecture, Repository Pattern y Unit of Work

Transferencia de conceptos de ASP.NET Core Web API (C#) a un ecosistema tecnológico alternativo

Material para aprendices
Trabajo en grupo · Entrega documentada

Contenido

Si la tabla aparece vacía, haga clic derecho sobre ella y seleccione «Actualizar campos» (o presione F9) en Word.

1. Objetivo general.....	4
2. Objetivos específicos	4
3. Lenguajes o tecnologías sugeridas	4
4. Prompts obligatorios para usar IA.....	5
5. Requisitos funcionales	5
5.1 Entidad Compañía.....	5
5.2 Entidad Empleado	5
5.3 Relación	6
6. Endpoints mínimos requeridos	6
6.1 Compañías	6
6.2 Empleados	6
7. Arquitectura obligatoria	6
7.1 Capa Domain	7
7.2 Capa Application o Services	7
7.3 Capa Infrastructure.....	7
7.4 Capa API o Presentation.....	8
8. Actividad paso a paso.....	8
Paso 1: Crear tabla comparativa con ASP.NET Core.....	8
Paso 2: Investigar el ORM	8
Paso 3: Crear el proyecto base	9
Paso 4: Crear las entidades	9
Paso 5: Configurar la base de datos	9
Paso 6: Crear migraciones.....	9
Paso 7: Insertar datos iniciales	10
Paso 8: Implementar Repository Pattern	10
Paso 9: Implementar Unit of Work	10
Paso 10: Crear la capa de servicios.....	11
Paso 11: Crear un caso transaccional obligatorio.....	12
Paso 12: Crear los endpoints REST	12
Paso 13: Validar rutas, nombres y binding.....	12
Paso 14: Agregar logging	13
9. Producto final esperado	13
10. README mínimo requerido.....	13
11. Criterios de evaluación	14

12. Preguntas para sustentación	14
13. Regla técnica obligatoria.....	15
Resultado esperado de aprendizaje	15

1. Objetivo general

Construir una API REST funcional en un lenguaje de programación diferente a C#, aplicando Onion Architecture, Repository Pattern y Unit of Work, y utilizando el ORM más popular o recomendado para ese ecosistema tecnológico.

Meta de aprendizaje: demostrar que los principios de arquitectura aprendidos en ASP.NET Core son transferibles a otros lenguajes y frameworks.

2. Objetivos específicos

Al finalizar la actividad, el aprendiz deberá ser capaz de:

- Investigar con apoyo de IA cómo se implementan en otro lenguaje los conceptos vistos en ASP.NET Core.
- Identificar el ORM más usado o recomendado para la tecnología asignada.
- Crear las entidades Compañía y Empleado.
- Configurar una relación uno a muchos entre ambas entidades.
- Implementar el Repository Pattern.
- Implementar o explicar técnicamente el patrón Unit of Work.
- Crear servicios de aplicación (capa de servicios).
- Crear endpoints REST funcionales.
- Agregar logging y manejo de errores.
- Documentar el proceso y justificar las decisiones técnicas.

3. Lenguajes o tecnologías sugeridas

Cada grupo trabajará con la tecnología asignada. La siguiente tabla resume las combinaciones recomendadas:

Lenguaje / Framework	ORM sugerido	Unit of Work
Python con FastAPI	SQLAlchemy	Session / transacciones
Python con Django	Django ORM	Transacciones con atomic
Java con Spring Boot	JPA / Hibernate	EntityManager / Transaction Manager
Node.js con NestJS	TypeORM o Prisma	Transaction Manager / Prisma Transaction
PHP con Laravel	Eloquent ORM	DB Transactions
Ruby on Rails	Active Record	Transactions
Kotlin con Spring Boot	JPA / Hibernate	Transaction Manager
Go	GORM	DB Transactions

Nota: cada grupo debe investigar si en su tecnología el patrón Unit of Work se implementa manualmente o si el ORM ya lo maneja internamente.

4. Prompts obligatorios para usar IA

Los aprendices deben guardar evidencia de sus consultas con IA. Reemplacen los campos entre corchetes por su tecnología y ORM asignados.

Prompt 1 — Comparación con ASP.NET Core

Estoy aprendiendo ASP.NET Core Web API con C#. Ahora debo implementar una API en [lenguaje/framework asignado]. Explicame cómo se traducen los conceptos de Controller, Entity, DbContext, Migration, Repository Pattern, Unit of Work, Service Layer, Dependency Injection y Logging.

Prompt 2 — ORM y Unit of Work

¿Cuál es el ORM más popular o recomendado para [lenguaje/framework asignado]? Explicame cómo maneja entidades, relaciones, migraciones, repositorios, transacciones y el patrón Unit of Work.

Prompt 3 — Arquitectura Onion

Propón una estructura de carpetas usando Onion Architecture en [lenguaje/framework asignado], incluyendo Domain, Application, Infrastructure y API. La solución debe usar Repository Pattern y Unit of Work.

Prompt 4 — Repository Pattern

Muéstrame cómo implementar Repository Pattern para las entidades Compañía y Empleado en [lenguaje/framework asignado].

Prompt 5 — Unit of Work

Explicame cómo implementar el patrón Unit of Work en [lenguaje/framework asignado] usando [ORM seleccionado]. Incluye cómo agrupar operaciones, manejar transacciones, confirmar cambios y revertir operaciones si ocurre un error.

Prompt 6 — Endpoints REST

Diseña los endpoints REST para administrar compañías y empleados. Incluye rutas, métodos HTTP, parámetros, body, códigos de respuesta y buenas prácticas REST.

5. Requisitos funcionales

5.1 Entidad Compañía

La entidad Compañía debe tener como mínimo los siguientes campos:

Campo	Descripción
Id	Llave primaria
Nombre	Nombre de la compañía
Direccion	Dirección física
Telefono	Número de contacto
FechaCreacion	Fecha de registro

5.2 Entidad Empleado

La entidad Empleado debe tener como mínimo los siguientes campos:

Campo	Descripción
Id	Llave primaria
Nombre	Nombre del empleado
Apellido	Apellido del empleado
Correo	Correo electrónico
Cargo	Cargo o rol
Salario	Salario asignado
CompaniaId	Llave foránea hacia Compañía

5.3 Relación

Compañía 1 — * Empleado

Una compañía puede tener muchos empleados. Cada empleado pertenece a una sola compañía.

6. Endpoints mínimos requeridos

6.1 Compañías

Método	Ruta	Descripción
GET	/api/companias	Listar todas las compañías
GET	/api/companias/{id}	Consultar una compañía por id
POST	/api/companias	Crear una compañía
PUT	/api/companias/{id}	Actualizar una compañía
DELETE	/api/companias/{id}	Eliminar una compañía
GET	/api/companias/{id}/empleados	Listar empleados de una compañía

6.2 Empleados

Método	Ruta	Descripción
GET	/api/empleados	Listar todos los empleados
GET	/api/empleados/{id}	Consultar un empleado por id
POST	/api/empleados	Crear un empleado
PUT	/api/empleados/{id}	Actualizar un empleado
DELETE	/api/empleados/{id}	Eliminar un empleado

7. Arquitectura obligatoria

La solución debe respetar una separación por capas siguiendo Onion Architecture:

- **Domain:** Núcleo del dominio: entidades e interfaces.

- **Application:** Servicios, casos de uso, DTOs y validaciones.
- **Infrastructure:** ORM, base de datos e implementaciones concretas.
- **API / Presentation:** Controladores, rutas, middlewares y logging.

7.1 Capa Domain

Debe contener entidades, reglas básicas del dominio, interfaces de repositorios e interfaz de Unit of Work (si la tecnología lo permite).

```
domain/  
  entities/  
    compania  
    empleado  
  interfaces/  
    compania_repository  
    empleado_repository  
    unit_of_work
```

7.2 Capa Application o Services

Debe contener servicios de aplicación, casos de uso, DTOs, validaciones y lógica de negocio. Los servicios deben depender de abstracciones, no directamente del ORM.

```
application/  
  services/  
    compania_service  
    empleado_service  
  dtos/  
    compania_dto  
    empleado_dto
```

Flujo esperado:

```
Controller / Route  
  ↓  
Service  
  ↓  
Unit of Work  
  ↓  
Repositories  
  ↓  
ORM / Database
```

7.3 Capa Infrastructure

Debe contener la configuración del ORM, la conexión a la base de datos, migraciones, implementación concreta de repositorios y de Unit of Work, y el manejo de transacciones.

```
infrastructure/  
  database/  
    connection  
    migrations  
    seed_data  
  repositories/  
    compania_repository_impl  
    empleado_repository_impl
```

```
unit_of_work/
  unit_of_work_impl
```

7.4 Capa API o Presentation

Debe contener controladores o rutas, endpoints, inyección de dependencias, middlewares, manejo global de errores y logging.

```
api/
  controllers/
    companias_controller
    empleados_controller
  routes/
  middlewares/
  main
```

8. Actividad paso a paso

Paso 1: Crear tabla comparativa con ASP.NET Core

Cada grupo debe completar la siguiente tabla, identificando el equivalente de cada concepto en su tecnología asignada:

Concepto en ASP.NET Core	Equivalente en la tecnología asignada
Controller	
Entity	
DbContext	
DbSet	
Migration	
Fluent API	
Repository	
Unit of Work	
Service Layer	
Dependency Injection	
appsettings.json	
Program.cs / Startup.cs	
Middleware	
Logging	

Paso 2: Investigar el ORM

Cada grupo debe responder las siguientes preguntas sobre el ORM seleccionado:

- ¿Cuál ORM van a usar?
- ¿Por qué ese ORM es adecuado?
- ¿Cómo se define una entidad?

- ¿Cómo se configura una relación uno a muchos?
- ¿Cómo se hacen migraciones?
- ¿Cómo se insertan datos iniciales?
- ¿Cómo se realizan consultas básicas?
- ¿Cómo maneja el ORM las transacciones?
- ¿El ORM implementa Unit of Work internamente?

Paso 3: Crear el proyecto base

Cada grupo debe crear un proyecto nuevo en la tecnología asignada y documentar:

- Lenguaje usado
- Versión del lenguaje
- Framework usado
- ORM instalado
- Motor de base de datos usado
- Comando usado para crear el proyecto
- Comando usado para instalar dependencias

Paso 4: Crear las entidades

Crear las entidades Compañía y Empleado. Cada entidad debe incluir:

- Llave primaria
- Campos obligatorios
- Relación uno a muchos
- Restricciones básicas
- Nombres adecuados para tablas y columnas

Paso 5: Configurar la base de datos

Configurar la cadena de conexión y documentar:

- Host
- Puerto
- Nombre de base de datos
- Usuario
- Contraseña
- Cadena de conexión
- Archivo de configuración usado

Si usan SQLite: deben indicar dónde se almacena el archivo de base de datos.

Paso 6: Crear migraciones

Deben generar y ejecutar migraciones, entregando evidencia de:

- Comando para crear migración
- Comando para aplicar migración

- Tablas generadas
- Relación entre Compañía y Empleado

Paso 7: Insertar datos iniciales

Deben insertar como mínimo:

- **3 compañías**
- **10 empleados**

Cada empleado debe estar relacionado con una compañía existente.

Paso 8: Implementar Repository Pattern

Crear repositorios para Compañía y Empleado. Cada repositorio debe tener métodos equivalentes a:

Método	Propósito
GetAll	Obtener todos los registros
GetById	Obtener un registro por id
Create	Crear un nuevo registro
Update	Actualizar un registro existente
Delete	Eliminar un registro
FindByCondition	Buscar por una condición

Importante: los repositorios no deben confirmar directamente los cambios en base de datos si se está usando Unit of Work.

Paso 9: Implementar Unit of Work

Este paso es obligatorio. Cada grupo debe investigar e implementar el patrón Unit of Work, o explicar técnicamente cómo el ORM lo proporciona.

¿Qué debe hacer Unit of Work?

- Coordinar varios repositorios.
- Controlar una única sesión o contexto de base de datos.
- Agrupar varias operaciones en una misma transacción.
- Confirmar todos los cambios juntos.
- Revertir cambios si ocurre un error.
- Evitar que cada repositorio maneje su propia transacción.

Ejemplo conceptual de Unit of Work

```
UnitOfWork
- CompaniaRepository
- EmpleadoRepository
- SaveChanges()
- BeginTransaction()
- Commit()
- Rollback()
```

O con nombres equivalentes según el lenguaje:

```
UnitOfWork
- companias
- empleados
- save()
- commit()
- rollback()
```

Flujo esperado usando Unit of Work

```
Service
↓
UnitOfWork
↓
Repositories
↓
ORM Context / Session
↓
Database
```

Ejemplo de caso de uso: crear una compañía, crear varios empleados para esa compañía y confirmar todos los cambios en una sola transacción. Si falla la creación de un empleado, no se guarda nada.

Preguntas obligatorias sobre Unit of Work

Cada grupo debe responder en su documentación:

1. ¿Qué es Unit of Work?
2. ¿Qué problema resuelve?
3. ¿Qué relación tiene con Repository Pattern?
4. ¿El ORM seleccionado ya implementa Unit of Work internamente?
5. ¿Qué objeto representa la unidad de trabajo? (DbContext en EF, Session en SQLAlchemy, EntityManager en JPA/Hibernate, Prisma Transaction en Prisma, DB Transaction en Laravel)
6. ¿Dónde se ubica Unit of Work dentro de Onion Architecture?
7. ¿Los repositorios llaman directamente a Save, Commit o Flush?
8. ¿Cómo se revierte una operación cuando ocurre un error?
9. ¿Cómo se garantiza que varias operaciones se guarden como una sola unidad?
10. ¿Qué ventajas tiene usar Unit of Work en una API empresarial?

Paso 10: Crear la capa de servicios

Crear servicios como `CompaniaService` y `EmpleadoService`. Los servicios deben usar Unit of Work.

```
CompaniaService
- get_all()
- get_by_id(id)
- create(dto)
- update(id, dto)
- delete(id)
```

Regla: el controlador no debe acceder directamente al ORM ni a los repositorios concretos.

Paso 11: Crear un caso transaccional obligatorio

Cada grupo debe implementar un endpoint o servicio que demuestre Unit of Work: crear una compañía con varios empleados en una sola operación.

Ruta sugerida: POST /api/companias/con-empleados

Body sugerido:

```
{
  "nombre": "Tech Solutions S.A.S",
  "direccion": "Calle 45 # 10-20",
  "telefono": "3001234567",
  "empleados": [
    {
      "nombre": "Ana",
      "apellido": "Gómez",
      "correo": "ana.gomez@tech.com",
      "cargo": "Desarrolladora",
      "salario": 3500000
    },
    {
      "nombre": "Carlos",
      "apellido": "Rojas",
      "correo": "carlos.rojas@tech.com",
      "cargo": "Tester",
      "salario": 2800000
    }
  ]
}
```

Condición: si falla la creación de un empleado, no debe guardarse la compañía ni ningún empleado. Esto debe demostrarse mediante una prueba.

Paso 12: Crear los endpoints REST

Implementar los endpoints mínimos (GET, POST, PUT, DELETE) y probarlos con alguna de estas herramientas:

- Swagger
- Postman
- Thunder Client
- Insomnia
- Herramienta equivalente

Paso 13: Validar rutas, nombres y binding

Cada grupo debe revisar:

- Que las rutas usen nombres REST correctos.
- Que los métodos HTTP sean apropiados.
- Que los parámetros por ruta funcionen.
- Que el body se reciba correctamente.
- Que las respuestas tengan códigos HTTP adecuados.

Código	Uso
200	Consulta exitosa
201	Recurso creado
204	Eliminación exitosa sin contenido
400	Error de validación
404	Recurso no encontrado
500	Error interno

Paso 14: Agregar logging

Cada aplicación debe incluir logging básico que registre:

- Inicio de la aplicación
- Creación de una compañía
- Creación de un empleado
- Inicio de una transacción
- Confirmación de una transacción
- Rollback de una transacción
- Errores de base de datos
- Errores inesperados

9. Producto final esperado

Cada grupo debe entregar:

1. Código fuente del proyecto.
2. Documento explicativo.
3. Capturas de pantalla de pruebas.
4. Evidencia del uso de IA.
5. Tabla comparativa con ASP.NET Core.
6. Diagrama de Onion Architecture.
7. Diagrama entidad-relación.
8. Diagrama del flujo Repository + Unit of Work.
9. Archivo README.md.
10. Base de datos creada o instrucciones para generarla.
11. Evidencia del endpoint transaccional.
12. Exposición corta del proyecto.

10. README mínimo requerido

El archivo README.md debe contener, como mínimo, las siguientes secciones:

```
# API de Compañías y Empleados
```

```

## Tecnología usada
## ORM usado
## Arquitectura aplicada
## Estructura del proyecto
## Entidades
## Relación entre entidades
## Repository Pattern
## Unit of Work
#### ¿Qué es Unit of Work?
#### ¿Cómo se implementó en esta tecnología?
#### ¿Cómo se manejan las transacciones?
#### ¿Cómo se hace commit?
#### ¿Cómo se hace rollback?
## Endpoints
## Endpoint transaccional
## Instalación
## Configuración de base de datos
## Migraciones
## Ejecución del proyecto
## Pruebas con Swagger/Postman
## Logging
## Uso de IA
## Conclusiones

```

11. Criterios de evaluación

Criterio	Puntaje
Investigación con IA y justificación técnica	10
Comparación con ASP.NET Core	10
Correcta aplicación de Onion Architecture	15
Entidades y relación Compañía-Empleado	10
Uso correcto del ORM	10
Migraciones y datos iniciales	10
Repository Pattern	10
Unit of Work y manejo transaccional	15
Endpoints REST funcionales	5
Logging y manejo de errores	5
TOTAL	100

12. Preguntas para sustentación

Cada aprendiz debe estar preparado para responder:

1. ¿Qué es Unit of Work?
2. ¿Qué problema resuelve?
3. ¿Cómo se relaciona Unit of Work con Repository Pattern?
4. ¿Dónde se ubica Unit of Work dentro de Onion Architecture?

5. ¿El ORM usado implementa Unit of Work internamente?
6. ¿Cómo se hace commit?
7. ¿Cómo se hace rollback?
8. ¿Qué pasa si una operación falla dentro de una transacción?
9. ¿Por qué no es recomendable que cada repositorio guarde cambios por separado?
10. ¿Cómo se implementó el endpoint transaccional?
11. ¿Qué similitudes encontró con DbContext de Entity Framework?
12. ¿Qué diferencias encontró frente a ASP.NET Core?

13. Regla técnica obligatoria

La aplicación debe demostrar claramente este flujo:

```
Controller / Route
    ↓
Service Layer
    ↓
Unit of Work
    ↓
Repositories
    ↓
ORM
    ↓
Database
```

No se acepta una solución donde:

- Controller → ORM directamente

Tampoco se acepta que cada repositorio confirme cambios de manera independiente sin explicar cómo se controla la transacción.

Resultado esperado de aprendizaje

Al finalizar, el aprendiz será capaz de transferir conceptos aprendidos en ASP.NET Core Web API con C# a otro ecosistema tecnológico, entendiendo que Onion Architecture, Repository Pattern, Unit of Work, ORM, migraciones, servicios, rutas, logging y transacciones son principios aplicables en múltiples lenguajes y frameworks.